

ОСНОВНЫЕ ТИПЫ ДАННЫХ

Четыре кита Python, с которых начинается любая программа

INT — ЦЕЛЫЕ ЧИСЛА

integer — числа без дробной части: **-5, 0, 42**

STR — СТРОКИ

string — текст в кавычках: **'привет'**

FLOAT — ДРОБНЫЕ ЧИСЛА

floating-point — числа с точкой: **3.14**

BOOL — БУЛЕВЫЙ ТИП

boolean — только **True** или **False**

ЦЕЛЫЕ ЧИСЛА INT

7 + 2	9	сложение
7 - 5	2	вычитание
8 * 2	16	умножение
6 / 3	2.0	деление (всегда float)
2 ** 5	32	степень
7 % 3	1	остаток от деления (MOD)
7 // 3	2	целое от деления (DIV)

% И // С ОТРИЦАТЕЛЬНЫМИ

17 % 4	1
17 // 4	4
(-17) % 4	3
(-17) // 4	-5

Важно! работают не так, как с положительными числами

ПЕРЕМЕННАЯ МЕНЯЕТ САМУ СЕБЯ

```
x = 10
x += 5 # x = 15

y = 10
y -= 7 # y = 3

z = 5
z *= 6 # z = 30
```

ПОЛЕЗНЫЕ ФУНКЦИИ С ЧИСЛАМИ

str(123) / int('123')	'123' / 123	число ↔ строка
int(17.9) / int(17.2)	17 / 17	отрубить дробную часть у float
round(17.9) / round(17.2)	18 / 17	арифметическое округление
abs(-20)	20	модуль числа
max(20, 40, 10, 30) / min(...)	40 / 10	максимум / минимум

СТРОКИ

STR

Текст, индексы и главный инструмент — срезы

ОДИНАКОВЫЕ СТРОКИ

```
# это одинаковые строки
s1 = 'Yo, informatics'
s2 = "Yo, informatics"
```

СИМВОЛЫ И ИНДЕКСЫ

```
s = 'INFAna100'
print(s[1]) # N
print(s[4]) # n
print(s[-3]) # 1
```

индексы идут слева направо с **0**, справа налево с **-1**

S[X : Y : N] — СРЕЗ СТРОКИ

начало среза

включительно, пусто → с начала строки

конец среза

НЕ включительно, пусто → до конца строки

шаг среза

каждый n-й символ, пусто → подряд

s[2:6:2]	'Fn'	со 2-го по 5-й вкл. через один
s[:6:2]	'IFn'	с начала по 5-й вкл. через один
s[2::2]	'Fn10'	со 2-го до конца через один
s[2:6]	'FAAna'	со 2-го по 5-й вкл. подряд
s[:6]	'INFAna'	с начала по 5-й вкл. подряд
s[2:]	'FAAna100'	со 2-го до конца подряд
s[-5:]	'na100'	с -5-го до конца подряд
s[:-3]	'INFAna'	с начала по -4-й вкл. подряд
s[::-1]	'001anAFNI'	перевернуть строку

СТРОКИ STR

МЕТОДЫ

s = 'Hello, World!' — главные методы строк

ВСЕ МЕТОДЫ НА ОДНОМ ПРИМЕРЕ

'Hello, ' + 'World!'	'Hello, World!'	сумма (конкатенация)
'x' * 25	'xxxx...x'	умножение строки на число
'l' in s / 'Goodbye' in s	True / False	находится ли символ/подстрока в строке
str(123) / int('123')	'123' / 123	превращение числа в строку и обратно
len(s)	12	длина строки (кол-во символов)
s.replace('l', '?')	'He??o, Wor?d!'	замена символа (можно указать сколько раз)
s.count('l')	3	сколько раз символ встречается
s.split() / s.split('o')	['Hello,', 'World!']	разделение строки по символу (по умолч. пробел)
' '.join(['Hello,', 'World!'])	'Hello, World!'	склеивание строки из элементов списка
s.index('l') / s.rindex('l')	2 / 9	индекс слева / справа
list('Hello')	['H', 'e', 'l', 'l', 'o']	строку в список

РАЗБОР ПО ШАГАМ

```
s = 'Hello, World!'
s = s.replace('l', '?') # 'He??o, Wor?d!'
parts = s.split(',') # разбить по запятой
back = '-'.join(parts) # склеить обратно через '-'
```

строки в Python **неизменяемы** — методы возвращают новую строку, исходную нужно переписать

ДРОБНЫЕ ЧИСЛА FLOAT & BOOL

Числа с точкой и логический тип

ДРОБНЫЕ ЧИСЛА FLOAT

```
x = 55.5
y = 123.3 # точки вместо запятых
```

с float можно делать всё то же самое, что и с int

странности округления

```
>>> 8 ** (1/3)
2.0
>>> 64 ** (1/3)
3.9999999999999996
```

float → int (отрубить дробную часть)

```
>>> int(55.9)
55
>>> int(6.22)
6
```

БУЛЕВЫЙ ТИП BOOL

```
m = True
n = False
```

```
a = 5 > 3
print(a) # True

b = 5 < 3
print(b) # False
```

Важно! bool'ы можно складывать и вычитать — они ведут себя как int (0 или 1)

```
>>> (2 > 1) + (7 >= 10) + (5 > 2)
2
>>> (2 > 1) + True + (5 > 2)
3
```

УСЛОВИЯ

IF / ELIF / ELSE

Ветвление программы по условиям

IF (ЕСЛИ)

```
score = 95
if score > 90:
    print('Топ!')
```

Если **score > 90**, выведется **Топ!**, иначе — ничего не происходит

IF + ELSE (ЕСЛИ + ИНАЧЕ)

```
score = 95
if score > 90:
    print('Топ!')
else:
    print('Не топ')
```

Если **score > 90**, выведется **Топ!**, иначе выведется **Не топ**

IF + ELIF + ELSE

```
score = 95
if score > 90:
    print('Топ!')
elif score > 80:
    print('Достойно')
else:
    print('Не топ')
```

Если **score > 90** → **Топ!**; если **80 < score ≤ 95** → **Достойно**; иначе → **Не топ**

elif может быть сколько угодно, не обязательно один

Можно соединять несколько условий в одно через **and** (И) и **or** (ИЛИ). **and** в приоритете выше, чем **or** (and как умножение, or как сложение)

ЦИКЛЫ FOR / WHILE

Повторение действий: по диапазону, по коллекции, по условию

БЕГ ПО ДИАПАЗОНУ — FOR I IN RANGE(X, Y, H)

начало цикла · конец цикла · шаг цикла

```
for i in range(3,10,2):  
    print(i)  
# 3, 5, 7, 9
```

```
for i in range(3,6):  
    print(i)  
# 3, 4, 5
```

```
for i in range(5):  
    print(i)  
# 0, 1, 2, 3, 4
```

БЕГ ПО СТРОКЕ / СПИСКУ — FOR C IN ...

```
for c in 'Hello':  
    print(c)  
# символы строки Hello  
# на разных строках
```

```
for x in [5,6,10,12]:  
    print(x)  
# 5, 6, 10, 12  
# на разных строках
```

WHILE <УСЛОВИЕ> (ПОКА)

```
x = 10  
y = 0  
while x < 30:  
    x += 6  
    y += 1  
print(x, y) # 34 4
```

Пока **x < 30**, цикл крутится:

x	y
10	0
16	1
22	2
28	3
34	4

СПИСКИ

LIST

Упорядоченный набор элементов с индексами

ЭЛЕМЕНТЫ И ИНДЕКСЫ

```
a1 = [6, 22, 71, 4, 28, 5]
a2 = [11, 'Hello', True, 55, 6.17]

a = [6, 22, 71, 4, 28, 5]
print(a[1])    # 22
print(a[4])    # 28
print(a[-3])   # 4
```

Список (list) — **упорядоченный** набор элементов, каждый со своим индексом

A[X : Y : N] — СРЕЗ СПИСКА

начало среза

включительно, пусто → с начала списка

конец среза

НЕ включительно, пусто → до конца списка

шаг среза

каждый n-й элемент, пусто → подряд

a[1:4:2]	[22, 4]	с 1-го по 3-й вкл. через один
a[:4:2]	[6, 71]	с начала по 3-й вкл. через один
a[1::2]	[22, 4, 5]	с 1-го до конца через один
a[1:4]	[22, 71, 4]	с 1-го по 3-й вкл. подряд
a[:4]	[6, 22, 71, 4]	с начала по 3-й вкл. подряд
a[1:]	[22, 71, 4, 28, 5]	с 1-го до конца подряд
a[-3:]	[4, 28, 5]	с -3-го до конца подряд
a[:-3]	[6, 22, 71]	с начала по -4-й вкл. подряд
a[::-1]	[5, 28, 4, 71, 22, 6]	перевернуть список

СПИСКИ LIST

МЕТОДЫ

`a = [10, 5, 20, -6]` — главные операции со списками

<code>a1+a2</code> (напр. <code>[10,5]+[20,-6]</code>)	<code>[10,5,20,-6]</code>	сумма (конкатенация)
<code>a * 2</code>	<code>[10,5,20,-6,10,5,20,-6]</code>	умножение списка на число
<code>10 in a</code> / <code>111 in a</code>	<code>True</code> / <code>False</code>	находится ли элемент в списке
<code>a.append(15)</code> / <code>a += [15]</code>	<code>[10,5,20,-6,15]</code>	добавить в конец 1 элемент
<code>a.append([15,100])</code>	<code>[...,15,100]</code>	добавить в конец набор элементов
<code>a.insert(2, 100)</code>	<code>[10,5,100,20,-6]</code>	добавить элемент в любое место
<code>a.remove(5)</code>	<code>[10,20,-6]</code>	удалить сам элемент (первый по счёту)
<code>a.pop(2)</code>	<code>[10,5,-6]</code>	удалить элемент по индексу
<code>a.sort()</code> / <code>sorted(a)</code>	<code>[-6,5,10,20]</code>	сортировка по возрастанию
<code>a.sort(reverse=True)</code>	<code>[20,10,5,-6]</code>	сортировка по убыванию
<code>a.index(20)</code> / <code>a.count(20)</code>	<code>2</code> / <code>1</code>	индекс элемента / сколько раз встречается
<code>len(a)</code> / <code>max(a)</code> / <code>min(a)</code>	<code>4</code> / <code>20</code> / <code>-6</code>	длина / максимум / минимум
<code>sum(a)</code>	<code>29</code>	сумма элементов списка

СТРОКИ ↔ СПИСКИ

<code>s.split()</code> / <code>s.split('o')</code>	<code>['Hello,', 'World!']</code>	разделение строки по символу
<code>' '.join(['Hello,', 'World!'])</code>	<code>'Hello, World!'</code>	склеивание строки из элементов списка
<code>list('Hello')</code>	<code>['H', 'e', 'l', 'l', 'o']</code>	строку в список

МНОЖЕСТВА

SET

Неупорядоченный набор уникальных элементов

СОЗДАНИЕ МНОЖЕСТВА

```
mn1 = {1, 8, 2, 4}
mn2 = set() # пустое множество
```

Главные отличия от списков: из множества **нельзя** доставать элементы по индексу; во множестве **не может быть** повторяющихся элементов

MN = {1, 8, 2, 4}

$\{1,2,8\} \& \{2,4,8,12\}$	$\{2, 8\}$	пересечение множеств
$\{1,2,8\} \mid \{2,4,8,12\}$	$\{1,2,4,8,12\}$	объединение множеств
$\{1,2,8\} - \{2,4,8,12\}$	$\{1\}$	вычитание множеств
<code>1 in mn</code> / <code>111 in mn</code>	<code>True</code> / <code>False</code>	находится ли элемент во множестве
<code>mn.add(15)</code>	$\{1,8,2,4,15\}$	добавить элемент
<code>mn.remove(2)</code>	$\{1,8,4\}$	удалить элемент
<code>sorted(mn)</code>	$[1,2,4,8]$	сортировка (превращает в список)
<code>len(mn)</code> / <code>max(mn)</code> / <code>min(mn)</code>	<code>4</code> / <code>8</code> / <code>1</code>	длина / максимум / минимум
<code>sum(mn)</code>	<code>15</code>	сумма элементов множества
<code>set([1,2,3,2,3,4])</code>	$\{1,2,3,4\}$	множество из списка — уникальные
<code>set('Hello')</code>	$\{'H','e','l','o'\}$	множество из строки — уникальные символы

ФУНКЦИИ

DEF / RETURN

Свой инструмент, который можно вызывать много раз

СИНТАКСИС

```
def имя_функции(аргумент1, аргумент2, ...):  
    тело функции  
    return результат
```

Важно! если в функции несколько **return**, выполняется самый первый из них, после чего функция завершает работу

ПРИМЕР — ФУНКЦИЯ НАХОЖДЕНИЯ ФАКТОРИАЛА

```
def factorial(x):  
    if x == 0:  
        return 1  
    proizv = 1  
    for i in range(1, x + 1):  
        proizv *= i  
    return proizv  
  
print(factorial(0))    # 1  
print(factorial(1))    # 1  
print(factorial(3))    # 6  
print(factorial(14))   # 87178291200
```

если сработал **return 1** внутри **if** — функция сразу завершит работу, не дойдя до цикла

ФУНКЦИЯ MAP & ПРИМЕРЫ

Обработка каждого элемента списка или строки

MAP(FUNC, OBJECT)

функция обработки · итерируемый объект — результат: объект **map**, его нужно завернуть в list/set или сразу найти sum/min/max

```
x = 1245
summ = sum(map(int, str(x))) # 12, сумма цифр числа
```

ФУНКЦИЯ ПОИСКА ДЕЛИТЕЛЕЙ

```
def dels(x):
    mn = set()
    for i in range(2, int(x**0.5)+1):
        if x%i==0:
            mn.add(i)
            mn.add(x//i)
    return mn
```

ФУНКЦИЯ ОПРЕДЕЛЕНИЯ ПРОСТОТЫ ЧИСЛА

```
def prost(x):
    return x>1 and all(x%i!= 0 for i in range(2, int(x**0.5) + 1))
```

оба шаблона перебирают делители только до $\sqrt{x}$ — этого достаточно, чтобы найти все пары делителей

СИСТЕМЫ СЧИСЛЕНИЯ

Перевод чисел между системами счисления

ИЗ ДЕСЯТИЧНОЙ В 2 / 8 / 16-ИЧНУЮ

<code>f'{28:b}' / bin(28)[2:]</code>	<code>'11100'</code>	в 2-ичную
<code>f'{28:o}' / oct(28)[2:]</code>	<code>'34'</code>	в 8-ичную
<code>f'{28:x}' / hex(28)[2:]</code>	<code>'1c'</code>	в 16-ичную

ИЗ ДЕСЯТИЧНОЙ В ЛЮБУЮ ДО 36

сами пишем функцию перевода числа x в систему счисления с основанием osn

```
from string import *

alf = '0123456789' + ascii_lowercase

def perevod(x, osn):
    s = ''
    while x > 0:
        ost = x % osn
        s = alf[ost] + s
        x = x // osn
    return s
```

<code>perevod(28, 7)</code>	<code>'40'</code>	в 7-ичную
<code>perevod(28, 8)</code>	<code>'34'</code>	в 8-ичную
<code>perevod(168, 16)</code>	<code>'a8'</code>	в 16-ичную
<code>perevod(164, 36)</code>	<code>'4k'</code>	в 36-ичную

ИЗ ЛЮБОЙ ДО 36 В ДЕСЯТИЧНУЮ

<code>int('11100', 2)</code>	<code>28</code>	из 2-ичной
<code>int('40', 7)</code>	<code>28</code>	из 7-ичной
<code>int('a8', 16)</code>	<code>168</code>	из 16-ичной
<code>int('4k', 36)</code>	<code>164</code>	из 36-ичной

ГЕНЕРАТОРЫ СПИСКОВ

Списочные выражения, all() и any()

[X FOR X IN RANGE(5) IF X%2==0]

что собираем
что?

где собираем
где?

при каком условии (необязательно)
когда?

ПРИМЕР — СОБРАТЬ ЧЁТНЫЕ ЭЛЕМЕНТЫ СПИСКА

```
a = [2,4,7,9,10,7]
a1 = [x for x in a if x % 2 == 0]
print(a1) # [2, 4, 10]
```

```
a = [2,4,7,9,10,7]
a1 = []
for x in a:
    if x % 2 == 0:
        a1.append(x)
# то же самое, но в цикле
```

ALL / ANY

```
a = [2,4,7,11,10,7]

# есть ли среди элементов чётные? кратные 3?
print(any(x % 2 == 0 for x in a)) # True
print(any(x % 3 == 0 for x in a)) # False

# все ли элементы больше 0? кратные 3?
print(all(x > 0 for x in a))      # True
print(all(x % 3 == 0 for x in a)) # False
```

РАБОТА С ФАЙЛАМИ И МОДУЛЬ RE

Чтение файлов и поиск по регулярным выражениям

РАБОТА С ФАЙЛАМИ

<code>f = open('777.txt')</code>	открыть файл
<code>f.readline()</code>	прочитать строку из файла
<code>f.readlines()</code>	прочитать все строки в список
<code>for s in f</code>	читать циклом строки в файле
<code>int(f.readline())</code>	прочитать число из файла
<code>a = [int(x) for x in f]</code>	прочитать числа из файла в список
<code>x, y = map(int, f.readline().split())</code>	прочитать пару чисел из файла

МОДУЛЬ RE (РЕГУЛЯРНЫЕ ВЫРАЖЕНИЯ)

```
from re import *
```

<code>finditer(match, s)</code>	поиск выражений match в строке s
<code>x.group()</code>	собрать найденный результат в строку
<code>[ABC] / [12345] / [+*/*]</code>	любой (один) из перечисленных символов
<code>[A-Z] / [1-5]</code>	любой (один) символ из диапазона
<code>.</code> (точка)	любой (один) произвольный символ
<code>*</code> (звёздочка)	повтор сколько угодно раз (может быть и пустота)
<code>+</code> (плюс)	повтор сколько угодно раз, но минимум один
<code>{5} / {5, 10}</code>	повтор ровно 5 раз / от 5 до 10 раз
<code>(AB) / (AB XY)</code>	поиск группы символов AB / AB или XY

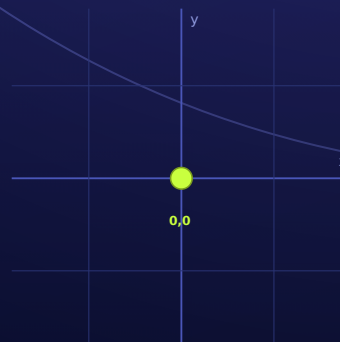
МОДУЛЬ TURTLE И IPADDRESS

Черепашья графика и работа с IP-адресами

МОДУЛЬ TURTLE

<code>from turtle import *</code>	импорт всех функций модуля
<code>forward(n) / fd(n)</code>	вперёд на n пикселей
<code>back(n) / bk(n)</code>	назад на n пикселей
<code>right(n) / rt(n)</code>	повернуться направо на n градусов
<code>left(n) / lt(n)</code>	аналогично, но влево
<code>penup() / up()</code>	поднять хвост
<code>pendown() / down()</code>	опустить хвост
<code>setpos(x,y) / goto(x,y)</code>	появиться в точке (x, y)
<code>dot() / dot(3,'blue')</code>	поставить точку (размер, цвет)
<code>tracer(0)</code>	отключить анимацию отрисовки
<code>screensize(2000,2000)</code>	размер окна (+прокрутка!)
<code>xcor() / ycor()</code>	текущие координаты
<code>exitonclick() / done()</code>	выйти по клику / не закрывать окно

ВОТ, ГДЕ ХОДИТ ЧЕРЕПАШКА



```
from turtle import *
tracer(0)
for i in range(4):
    forward(100)
    right(90)
done()
```

МОДУЛЬ IPADDRESS

<code>from ipaddress import *</code>	импорт всех функций модуля
<code>ip_address('128.198.15.111')</code>	создание объекта ip-адрес
<code>int(ip)</code>	перевод ip-адреса в десятичное число
<code>f'{ip:b}' / bin(int(ip))[2:].zfill(32)</code>	перевод ip-адреса в двоичное число
<code>ip_network('ip узла или сети/маска', 0)</code>	создание сети
<code>nn.netmask</code>	маска сети nn
<code>nn.num_addresses</code>	кол-во ВСЕХ адресов сети (вкл. служебные)
<code>list(nn.hosts())</code>	список IP узлов сети без служебных

МОДУЛЬ ITERTOOLS И MATH

Перестановки, комбинации и математические функции

PRODUCT — КАЖДЫЙ С КАЖДЫМ

```
from itertools import *

alf = 'ABCD'
for x in product(alf, repeat=3):
    s = ''.join(x)
    print(s)
# AAA, AAB, AAC, AAD, ABA...
```

repeat — длина конечной строки

PERMUTATIONS — ПЕРЕСТАНОВКИ

```
from itertools import *

alf = 'ABCD'
for x in permutations(alf, 3):
    s = ''.join(x)
    print(s)
# ABC, ABD, ACB, ACD, ADB...
```

МОДУЛЬ MATH

```
from math import *
```

импорт всех функций модуля math

<code>sqrt(16)</code>	4.0	квадратный корень
<code>log2(32)</code>	5.0	логарифм с основанием 2
<code>prod([1,2,3,7])</code>	42	произведение элементов списка
<code>ceil(7.7) / ceil(7.2)</code>	8 / 8	округлить вверх
<code>floor(7.7) / floor(7.2)</code>	7 / 7	округлить вниз
<code>factorial(5)</code>	120	факториал
<code>dist([-1,-1], [2,3])</code>	5.0	расстояние между точками